Category C: Encapsulation Practice Solutions

STEP SEM-3 · CodInClub | Java solutions for all 5 problems

#	Problem	File	Key idea
1	The Piggy Bank	PiggyBank.java	private state + final ID
2	The Quiz Scorecard	Scorecard.java	private array, expose score only
3	The Nickname Tag	NameTag.java	immutable, split once
4	The Locker Code	Locker.java	write-only field
5	The Attendance Sheet	AttendanceSheet.java	private array, count + lookup

All code was compiled and run; the outputs shown below are the real program output and match each problem's expected behavior. Run it with `javac *.java && java Main`.

Problem 1: The Piggy Bank

Money can only change through **deposit** and **withdraw**. The savings field is private, the ID is `final`, and a withdrawal larger than the savings is rejected.

Why it works

• **private int savings**: no setter exists, so it can only move through deposit/withdraw.
• **private final String id**: assigned once in the constructor and can never change.
• **withdraw** returns false and changes nothing when the amount exceeds savings.
• **getSavings()** is read-only access; there is no way to set the amount directly.

Solution: PiggyBank.java

```
public class PiggyBank {
    private final String id;      // fixed at creation
    private int savings;          // private: only deposit/withdraw change it

    public PiggyBank(String id) {
        this.id = id;
        this.savings = 0;
    }

    public void deposit(int amount) {
        if (amount > 0) {
            savings += amount;
        }
    }

    public boolean withdraw(int amount) {
        if (amount <= 0 || amount > savings) {
            return false;         // rejected, savings unchanged
        }
        savings -= amount;
        return true;
    }

    public int getSavings() {     // read-only access
        return savings;
    }

    public String getId() {
        return id;
    }
}
```

Output

```
start -> savings = 0
deposit(100) -> savings = 100
withdraw(30) -> savings = 70
withdraw(500) -> rejected, savings stays 70
```

Problem 2: The Quiz Scorecard

Results live in a private array filled one answer at a time. Only the computed score is exposed, never the array itself.

Why it works

- **private final boolean[] results**: size fixed at construction, never returned.
- **recorded** counter tracks where the next answer goes.
- **recordAnswer** rejects answers beyond the fixed question count.
- **getScore()** loops over the array internally and returns only a number.

Solution: Scorecard.java

```
public class Scorecard {
    private final boolean[] results;   // never exposed
    private int recorded = 0;          // how many answers recorded so far

    public Scorecard(int totalQuestions) {
        this.results = new boolean[totalQuestions];
    }

    public boolean recordAnswer(boolean correct) {
        if (recorded >= results.length) {
            return false;              // more answers than questions: rejected
        }
        results[recorded++] = correct;
        return true;
    }

    public int getScore() {
        int score = 0;
        for (int i = 0; i < recorded; i++) {
            if (results[i]) {
                score++;
            }
        }
        return score;
    }
}
```

Output

```
getScore() -> 3
recordAnswer(true) beyond 4 -> rejected, score still 3
```

Problem 3: The Nickname Tag

An immutable class: the full name is split once in the constructor, everything is stored in `final` fields, and there are no setters.

Why it works

• The class is **final** and all fields are **private final**.
• **split(" ")** runs once, in the constructor, and the nickname is built there.
• **getNickname()** just returns the stored result (e.g. "Maria G.").
• Two NameTags from the same name behave identically but are separate objects.

Solution: NameTag.java

```
public final class NameTag {
    private final String firstName;
    private final String lastName;
    private final String nickname;

    public NameTag(String fullName) {
        String[] parts = fullName.split(" ");   // split once, here
        this.firstName = parts[0];
        this.lastName = parts[1];
        this.nickname = firstName + " " + lastName.charAt(0) + ".";
    }

    public String getNickname() {
        return nickname;
    }
}
```

Output

```
getNickname() -> "Maria G."
```

Problem 4: The Locker Code

The combination can be changed but never read back: it is private with no getter, and **changeCode** demands the correct current code first.

Why it works

• **private String code** has no getter at all (write-only).
• **changeCode** checks the old code first; on a mismatch it returns false and touches nothing.
• **private final int lockerNumber** is fixed at creation.

Solution: Locker.java

```
public class Locker {
    private final int lockerNumber;   // fixed at creation
    private String code;              // private, no getter at all

    public Locker(int lockerNumber, String code) {
        this.lockerNumber = lockerNumber;
        this.code = code;
    }

    public boolean changeCode(String currentCode, String newCode) {
        if (!code.equals(currentCode)) {
            return false;             // wrong current code: do nothing else
        }
        code = newCode;
        return true;
    }

    public int getLockerNumber() {
        return lockerNumber;
    }
}
```

Output

```
changeCode("1234", "5678") -> success
changeCode("0000", "9999") -> rejected, code is still "5678"
changeCode("5678", "4321") -> success (confirms code was 5678)
```

Problem 5: The Attendance Sheet

Present students are stored in a private array. Only a count and a yes/no lookup are exposed, never the list.

Why it works

• **private final String[] present** with a fixed maximum size; never returned.
• **markPresent** checks **isPresent** first, so duplicates are ignored.
• **getPresentCount()** and **isPresent()** loop over the array internally only.

Solution: AttendanceSheet.java

```
public class AttendanceSheet {
    private final String[] present;   // never exposed
    private int count = 0;

    public AttendanceSheet(int maxClassSize) {
        this.present = new String[maxClassSize];
    }

    public boolean markPresent(String name) {
        if (isPresent(name)) {
            return false;             // no duplicates
        }
        if (count >= present.length) {
            return false;             // class full
        }
        present[count++] = name;
        return true;
    }

    public int getPresentCount() {
        return count;
    }

    public boolean isPresent(String name) {
        for (int i = 0; i < count; i++) {
            if (present[i].equals(name)) {
                return true;
            }
        }
        return false;
    }
}
```

Output

```
getPresentCount() -> 2
isPresent("Ben") -> true
isPresent("Chen") -> false
```

Test driver: Main.java

Runs the sample input/output from every problem.

```
public class Main {
    public static void main(String[] args) {
        System.out.println("--- Problem 1: PiggyBank ---");
        PiggyBank pb = new PiggyBank("PB-1");
        System.out.println("start -> savings = " + pb.getSavings());
        pb.deposit(100);
        System.out.println("deposit(100) -> savings = " + pb.getSavings());
        pb.withdraw(30);
        System.out.println("withdraw(30) -> savings = " + pb.getSavings());
        boolean ok = pb.withdraw(500);
        System.out.println("withdraw(500) -> " + (ok ? "applied" : "rejected")
                + ", savings stays " + pb.getSavings());

        System.out.println("--- Problem 2: Scorecard ---");
        Scorecard sc = new Scorecard(4);
        sc.recordAnswer(true);
        sc.recordAnswer(true);
        sc.recordAnswer(false);
        sc.recordAnswer(true);
        System.out.println("getScore() -> " + sc.getScore());
        System.out.println("recordAnswer(true) beyond 4 -> "
                + (sc.recordAnswer(true) ? "accepted" : "rejected")
                + ", score still " + sc.getScore());

        System.out.println("--- Problem 3: NameTag ---");
        NameTag tag = new NameTag("Maria Gomez");
        System.out.println("getNickname() -> \"" + tag.getNickname() + "\"");

        System.out.println("--- Problem 4: Locker ---");
        Locker l = new Locker(101, "1234");
        System.out.println("changeCode(\"1234\", \"5678\") -> "
                + (l.changeCode("1234", "5678") ? "success" : "rejected"));
        System.out.println("changeCode(\"0000\", \"9999\") -> "
                + (l.changeCode("0000", "9999") ? "success" : "rejected, code is still \"5678\""));
        System.out.println("changeCode(\"5678\", \"4321\") -> "
                + (l.changeCode("5678", "4321") ? "success (confirms code was 5678)" : "rejected"));

        System.out.println("--- Problem 5: AttendanceSheet ---");
        AttendanceSheet sheet = new AttendanceSheet(30);
        sheet.markPresent("Ana");
        sheet.markPresent("Ben");
        sheet.markPresent("Ana");
        System.out.println("getPresentCount() -> " + sheet.getPresentCount());
        System.out.println("isPresent(\"Ben\") -> " + sheet.isPresent("Ben"));
        System.out.println("isPresent(\"Chen\") -> " + sheet.isPresent("Chen"));
    }
}
```